Lecture 11: Adaptation

Kai-Wei Chang
CS @ UCLA
kw@kwchang.net

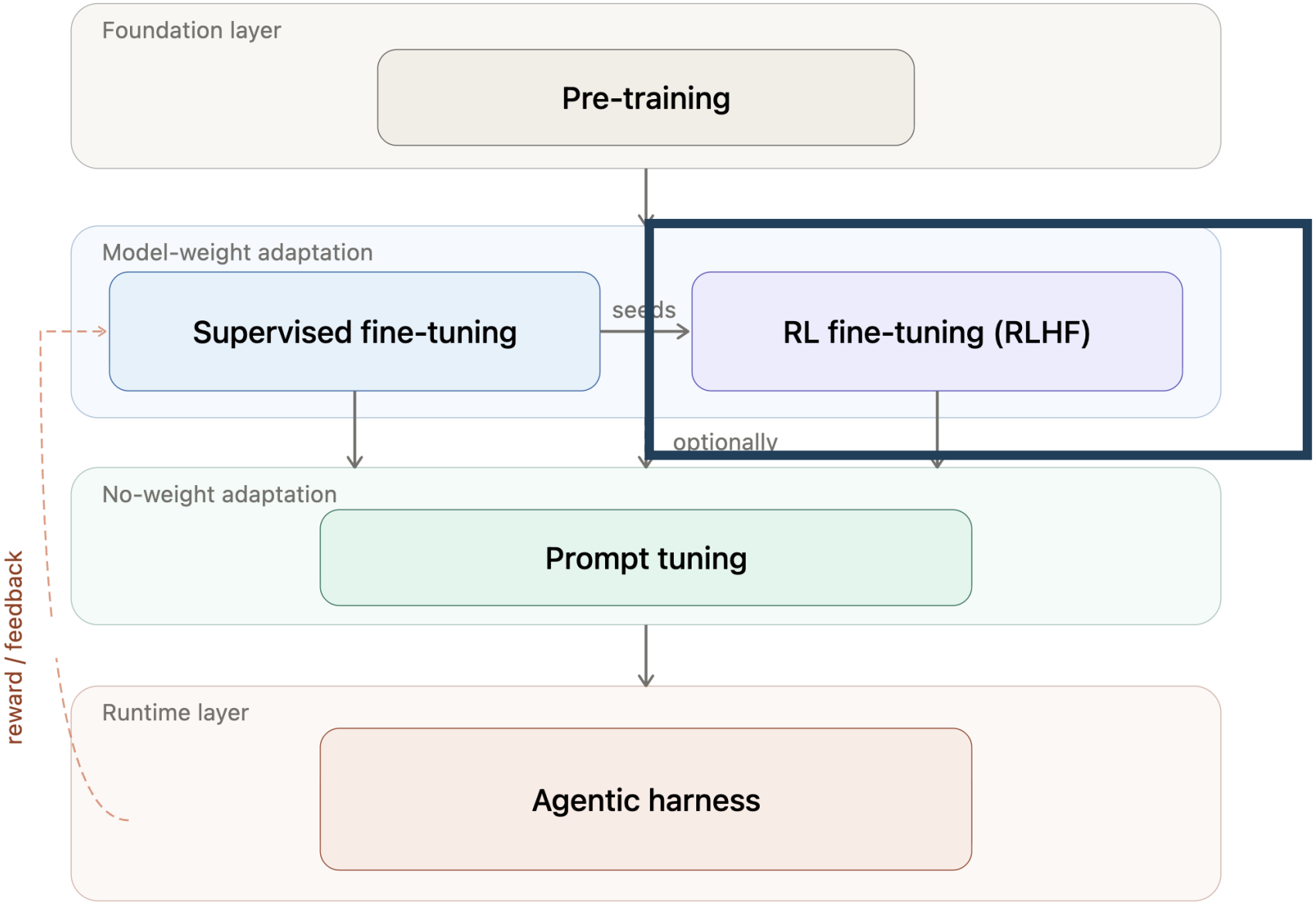
Credit: some slides are modified from Vivek Srikumar, Jason Eisner, Chris Manning

Announcement

- Quiz 4 due today
- Final Exam: 5/11
  - One question set about first question of Hw2
- Midterm report for final project: extend to 5/11
- Peer Review for Hw1
  - Due 5/15

Outline

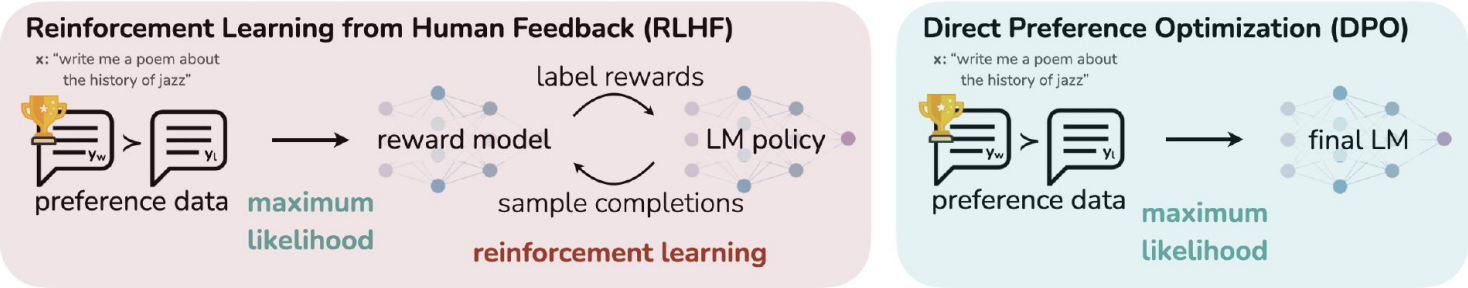
- Preference ranking – Direct Policy Optimization
- Parameter-efficient Adatation
  - Adapters
  - LoRA


Foundation layer
Pre-training
Model-weight adaptation
Supervised fine-tuning
seeds
RL fine-tuning (RLHF)
optionally
No-weight adaptation
Prompt tuning
reward / feedback
Runtime layer
Agentic harness

Direct Policy Optimization (DPO)

Direct Policy Optimization (DPO)

- Adopt an alternative ***offline RL*** setup
  - Offline RL uses a static set of trajectories with rewards, rather than new trajectories generated during learning (like we saw in REINFORCE and PPO)
- Restrict the reward to a specific form
- Combine the reward learning objective with an RL objective to directly optimize a *policy*
  - Recall: what's our *policy* here?



DPO Objectives

- DPO starts with a very similar RL objective to PPO

$$\arg\max_\theta E_{\bar{x}\sim\mathcal{D},\bar{y}\sim\pi_\theta(\bar{y}|\bar{x})}\left[r(\bar{x},\bar{y}) - \beta \mathrm{KL}[\pi_\theta(\bar{y}\,|\,\bar{x}), \pi_{\text{ref}}(\bar{y}\,|\,\bar{x})]\right]$$

Maximize the expected reward according to our prompt data and policy

Penalize for the distribution getting further from the pre-RL distribution

Recall PPO:

$$\arg\max_\theta E_\pi\left[\frac{\pi_\theta(a_t\,|\,s_t)}{\pi_{\theta_{\text{old}}}(a_t\,|\,s_t)}\hat{A}(s,a) - \beta \mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot\,|\,s_t), \pi_\theta(\cdot\,|\,s_t)]\right]$$

DPO Derivation

- We can express the reward function:

$$r(\bar{x}, \bar{y}) = \beta \log \frac{\pi^*(\bar{y} \mid \bar{x})}{\pi_{\text{ref}}(\bar{y} \mid \bar{x})} + \beta \log Z(\bar{x})$$

- Why is this important?
- Remember: the RLHF reward is the scoring function s(.) in the Bradley-Terry preference model
  - $p(a \succ b) = \sigma(s(a) - s(b))$

DPO Derivation

- We can express the reward function:

$$r(\bar{x}, \bar{y}) = \beta \log \frac{\pi^*(\bar{y} \mid \bar{x})}{\pi_{\text{ref}}(\bar{y} \mid \bar{x})} + \beta \log Z(\bar{x})$$

- So we can simply plug the above reward to the Bradley-Terry model:

$$\begin{aligned} p(\bar{y}_w \succ \bar{y}_l) &= \sigma\left(\beta \log \frac{\pi^*(\bar{y}_w \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_w \mid \bar{x})} + \beta \log Z(\bar{x}) - \beta \log \frac{\pi^*(\bar{y}_l \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_l \mid \bar{x})} - \beta \log Z(\bar{x})\right) \\ &= \sigma\left(\beta \log \frac{\pi^*(\bar{y}_w \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_w \mid \bar{x})} - \beta \log \frac{\pi^*(\bar{y}_l \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_l \mid \bar{x})}\right) \end{aligned}$$

DPO Derivation

- If we use π_θ instead of π^*, we directly get a negative log-likelihood loss to optimize:

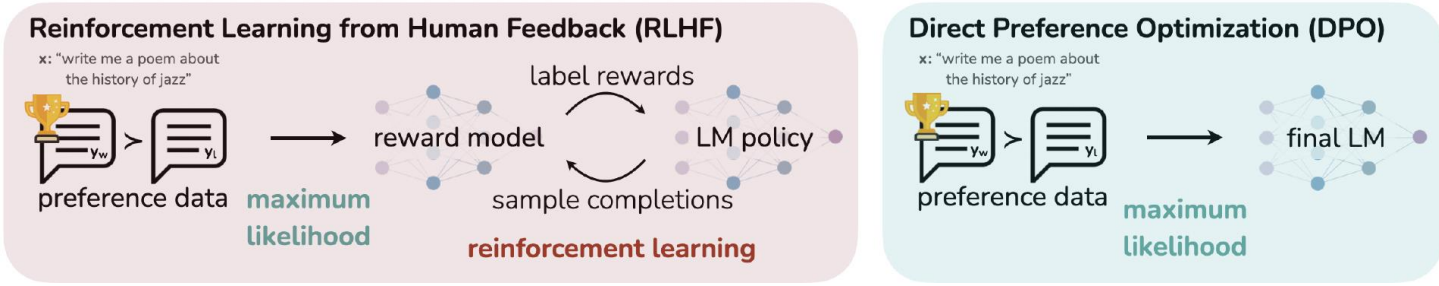
$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \prod_{(\bar{x},\bar{y}_w,\bar{y}_l)\in\mathcal{D}} p(y_w \succ y_l)$$

$$= -E_{(\bar{x},\bar{y}_w,\bar{y}_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(\bar{y}_w \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_w \mid \bar{x})} - \beta \log \frac{\pi_\theta(\bar{y}_l \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_l \mid \bar{x})}\right)\right]$$

Increase likelihood of preferred example

Decrease likelihood of dispreferred example

Comparing DPO to PPO



- RLHF (PPO) and DPO both can be used to compute rewards
  - RLHF (PPO): explicitly learns a reward model $r_\psi(x, y)$
  - DPO: we can compute the reward using the base and fine-tuned models $r(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$
- Reward model is a key, and there is still a lot of room for improvement (as is shown in Llama 2 results)

LLMs Alignment Key Takeaways

- RLHF is an essential, but complex and compute-intensive process to make expressive LLMs useful as multitasking agents.
- It presents a very restricted instance of RL (basically a bandit problem), even though it uses relatively advanced algorithms
- *Data is the key to the process*, and it requires careful curation and annotation
- There are supervised (offline RL) approaches that can get similar or even equal results (e.g., DPO)
- A lot of active research in this area

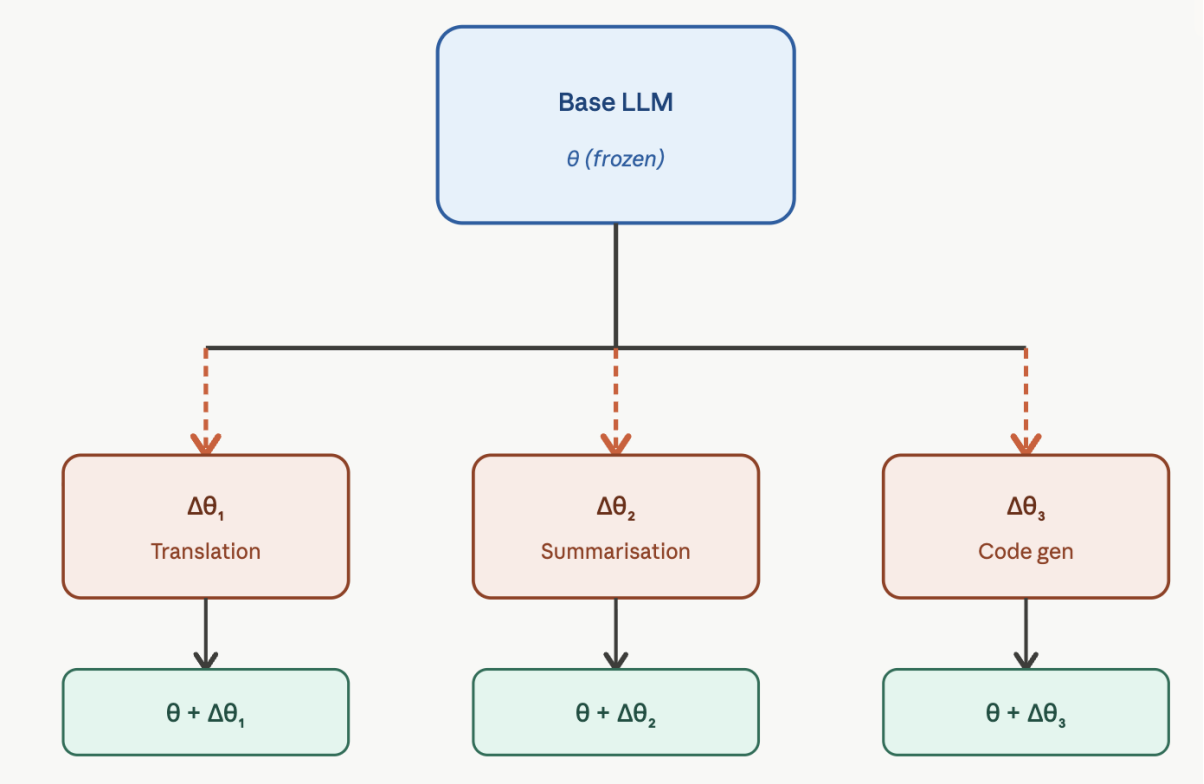
Parameter-Efficient Adaptation

Ideas for Parameter-Efficient Adaptation

- Consider we have an LLM $P_\theta(y|x)$
- Modern LLMs has 1B, 8B, 13B, >100B parameters θ (collections of weigh matrices W for query, value, key)

Ideas for Parameter-Efficient Adaptation

- We can have a pre-trained model θ_0 then train adaptor $\Delta\theta$ for different tasks by fixing θ_0
- Adapted LLM: $P_{\theta_0+\Delta\theta}(y|x)$



Adaptor

- We can fine-tune (SFT or RL) on the task data to update θ_0 to $\theta_0 + \Delta\theta$
  - Full fine-tuning: $|\Delta\theta| = |\theta_0|$

- Challenges of full fine-tuning?

Motivation

- Reduce cost
  - Fine-tune all parameters are impractical: computation, GPU memory, disk to store adaptors
- LLM are over-parameterized, fine-tune a subset of parameters can achieve the same performance
  - Lottery Ticket Hypothesis, Frankle & Carbin (2019) (see later slide)

The Lottery Ticket Hypothesis

Inside a large, randomly initialized neural network, there exists a much smaller subnetwork (“winning ticket”) that, if trained in isolation, can reach similar performance as the full model -- Frankle & Carbin (2019)

- Has been emperically verified in RL/LLM/CV areas

Ideas for Parameter-Efficient Adaptation

- Idea 1: Only update certain layers
  - Which layer to update?
- Idea 2: Only update a subnetwork
  - Design a sparse architecture
    - Activation sparsity: Mixture of Experts
    - Parameter sparsity: Pruning
- Idea 3: Low-rank Adapator

Recap: on Layer

$$Q = H^{l-1}W_l^Q$$
$$K = H^{l-1}W_l^K$$
$$V = H^{l-1}W_l^V$$

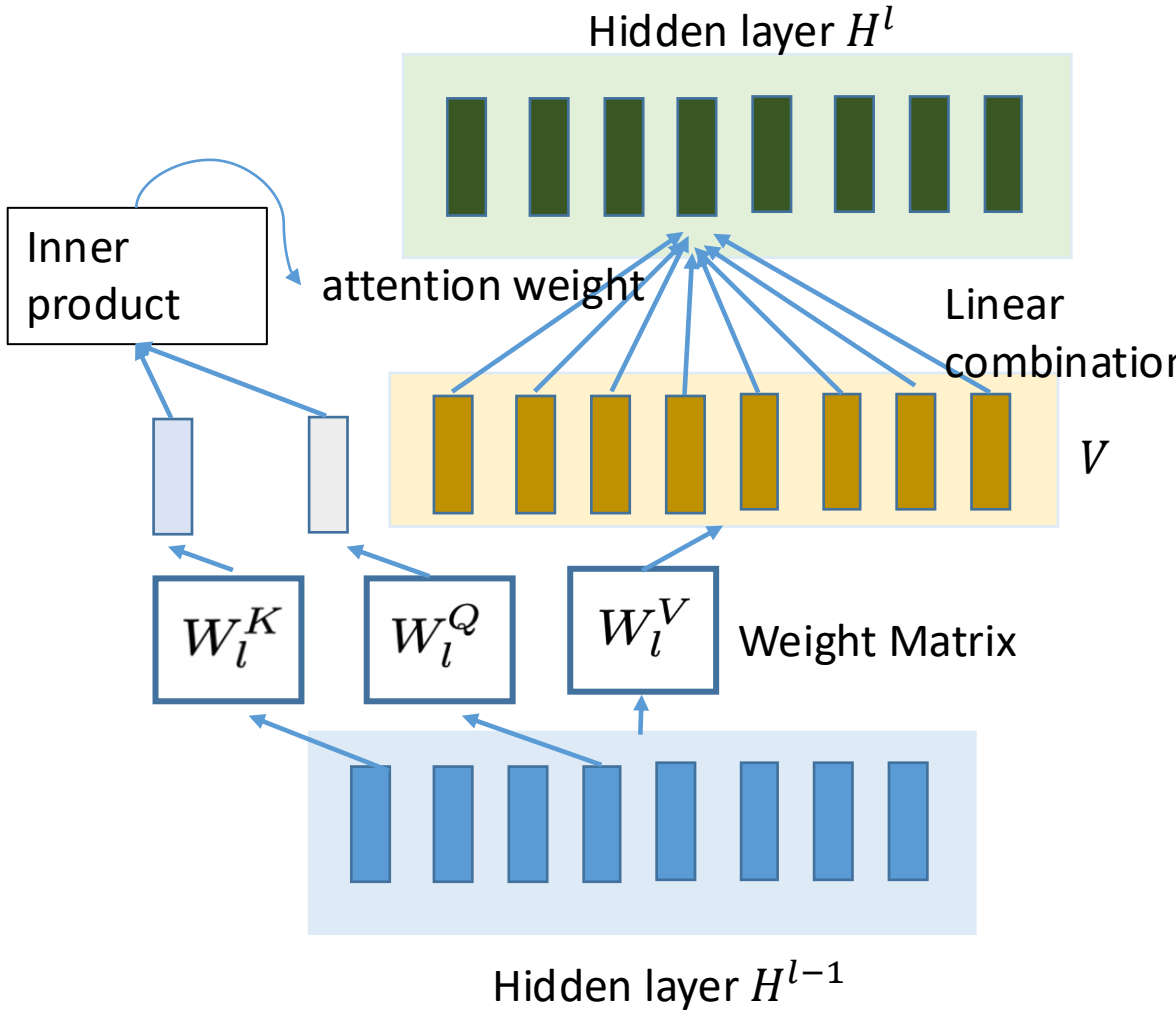
Query

Key

Value

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

a constant



https://poloclub.github.io/transformer-explainer/

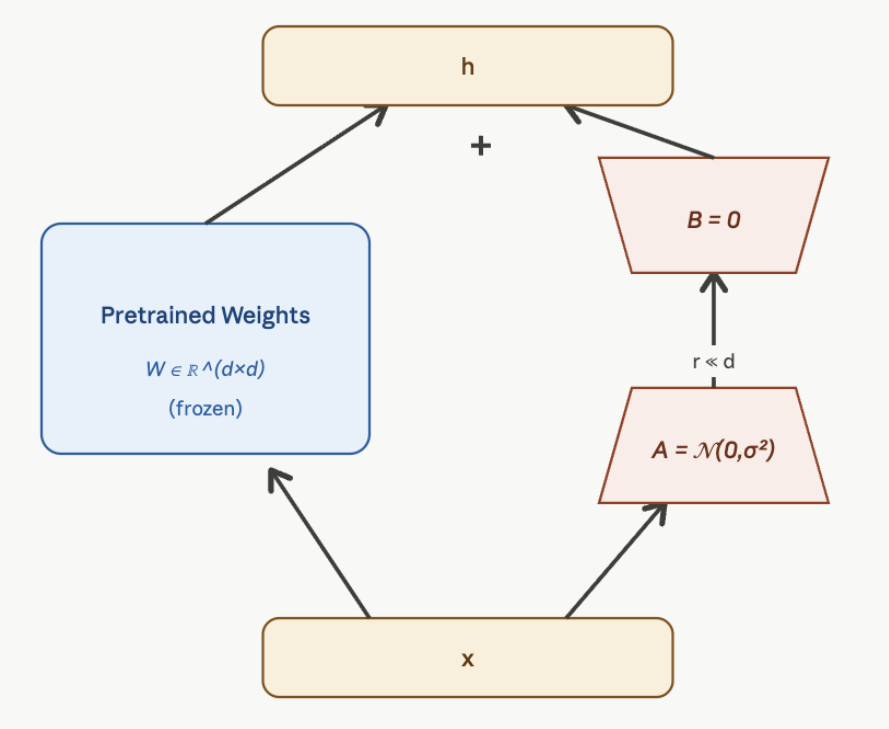
Low rank decomposition (LoRA)

- Update with a lower rank parameter matrix
- $W_0 \in \mathbb{R}^{d \times k}$: pre-trained weight matrix

$$W_0 + \Delta W = W_0 + \alpha BA$$

$$B \in \mathbb{R}^{d \times r}, A \times \mathbb{R}^{r \times k}$$

α: a hyper-parameters



Exercise

- $W_0 \in \mathbb{R}^{1000\times1000}$: pre-trained weight matrix

$$W_0 + \Delta W = W_0 + \alpha BA; \quad B \in \mathbb{R}^{d\times r}, A \times \mathbb{R}^{r\times k}$$

Assume r=4

How many parameters we save?

Exercise

- $W_0 \in \mathbb{R}^{1000\times1000}$: pre-trained weight matrix

$W_0 + \Delta W = W_0 + \alpha BA; \quad B \in \mathbb{R}^{d\times r}, A \times \mathbb{R}^{r\times k}$

Assume r=4

How many parameters we save?

full update: 1M parameters

LoRA: 1000×4×2 = 8,000

125x more parameter efficient

Example of Implementation

```
input_dim = 768  # e.g., the hidden size of the pre-trained model
output_dim = 768  # e.g., the output size of the layer
rank = 8  # The rank 'r' for the low-rank adaptation

W = ... # from pretrained network with shape input_dim x output_dim

W_A = nn.Parameter(torch.empty(input_dim, rank)) # LoRA weight A
W_B = nn.Parameter(torch.empty(rank, output_dim)) # LoRA weight B

# Initialization of LoRA weights
nn.init.kaiming_uniform_(W_A, a=math.sqrt(5))
nn.init.zeros_(W_B)

def regular_forward_matmul(x, W):
    h = x @ W
return h

def lora_forward_matmul(x, W, W_A, W_B):
    h = x @ W  # regular matrix multiplication
    h += x @ (W_A @ W_B)*alpha # use scaled LoRA weights
return h
```

Credit to https://lightning.ai/pages/community/article/lora-llm/

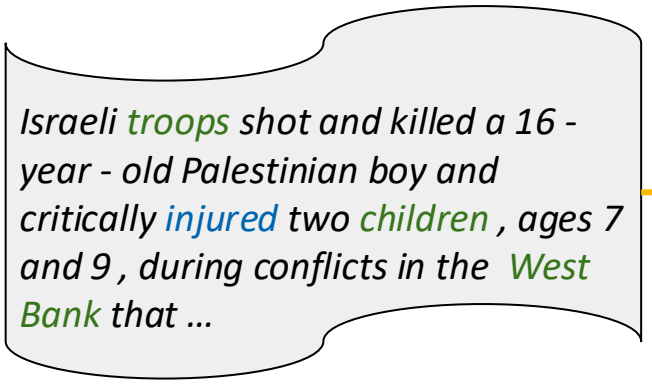
Empirical Results

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

Prompt Tuning

Event Extraction

- An event has its own type (*event type*) and its participants (*arguments*). An *event trigger* is the word that most clearly expresses the an event's occurrence. For each event type, there are a few designated *argument roles* to fill in.



Event Type	Life: Injure
Trigger	injured
Agent	troops
Victim	children
Instrument	-
Place	West Bank

Subtasks: (a) Trigger detection, (b) Argument detection, (c) Argument role prediction, (d) Event type classification

Language Generation for Event Extraction

Event Trigger	*detonated*
Attacker	*Palestinian*
Target	*jeep, soldiers*
Instrument	*bomb*
Place	*Gaza Strip*

Decode the output sequence into final predictions

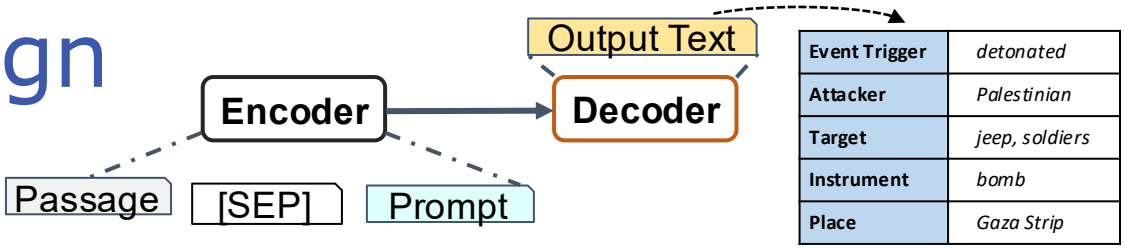
Autoregressive generation considers dependencies

Output

Generative Model

Earlier Monday , a 19-year-old *Palestinian* riding a bicycle *detonated* a 30-kilo (66-pound) *bomb* near a military *jeep* in the *Gaza Strip* , injuring three *soldiers*.

Prompt Design



Passage: *Earlier Monday , a 19-year-old Palestinian riding a bicycle detonated a 30-kilo (66-pound) bomb near a military jeep in the Gaza Strip , injuring three soldiers.*

Prompt

Can be obtained from annotation guideline

Event Type Description	*The event is related to conflict and some violent physical act.*
Event Keywords	*Similar triggers such as war, attack, terrorism.*
E2E Template	*Event trigger is <Trigger>. \n some attacker attacked some facility, someone, or some organization by some way in somewhere.*

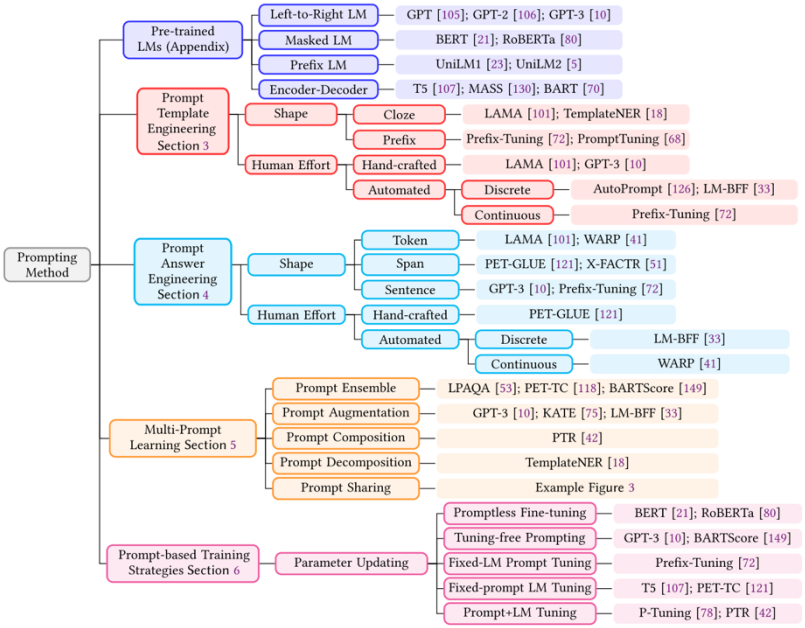
Can be obtained from training data.

Output Text

Event trigger is detonated. \n Palestinian attacked jeep and soldiers by bomb in Gaza Strip.

Consideration of Prompt Design

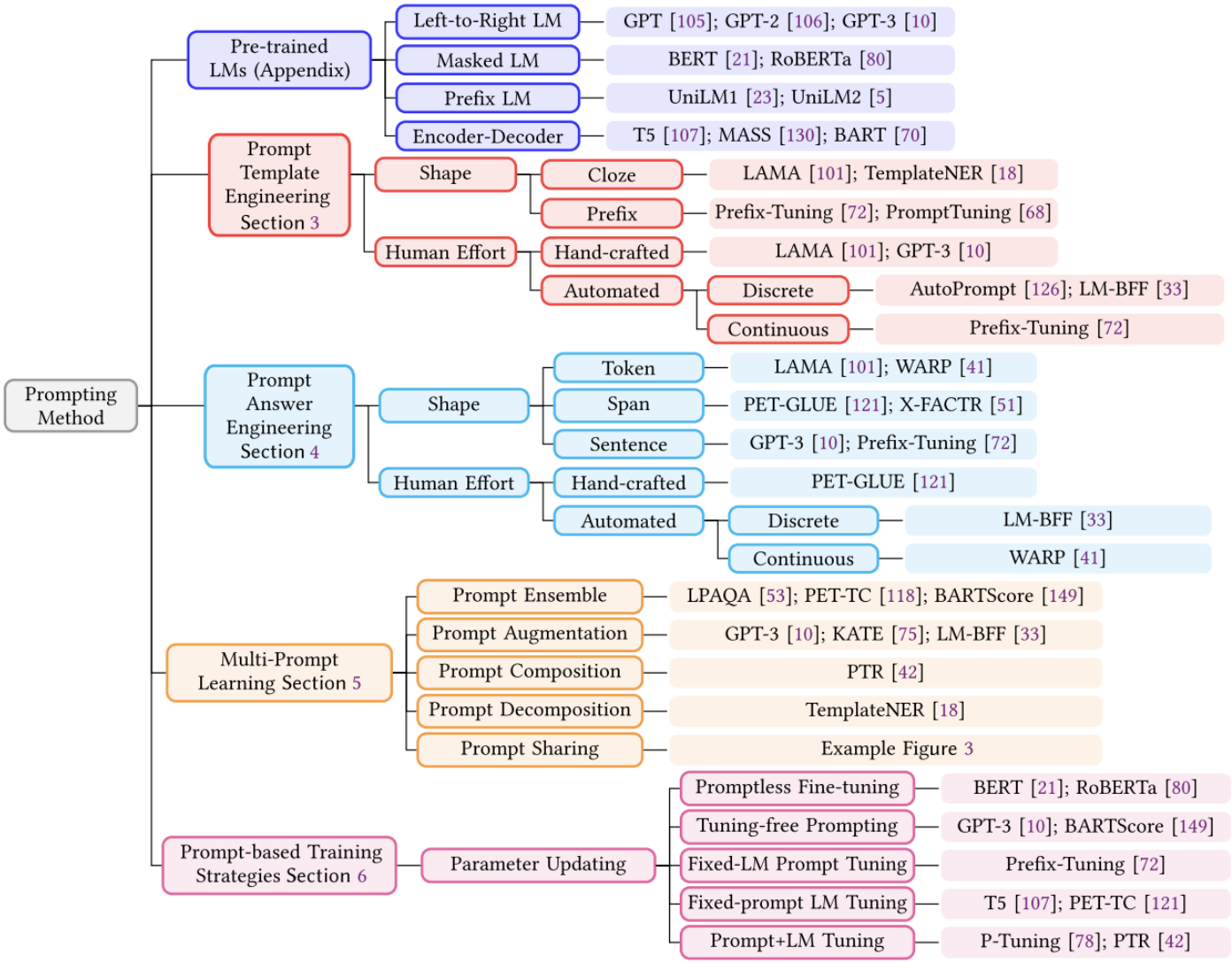
- Base language model
- Prompt template
- Answer mapping
- Prompt expansion
- Learning protocol



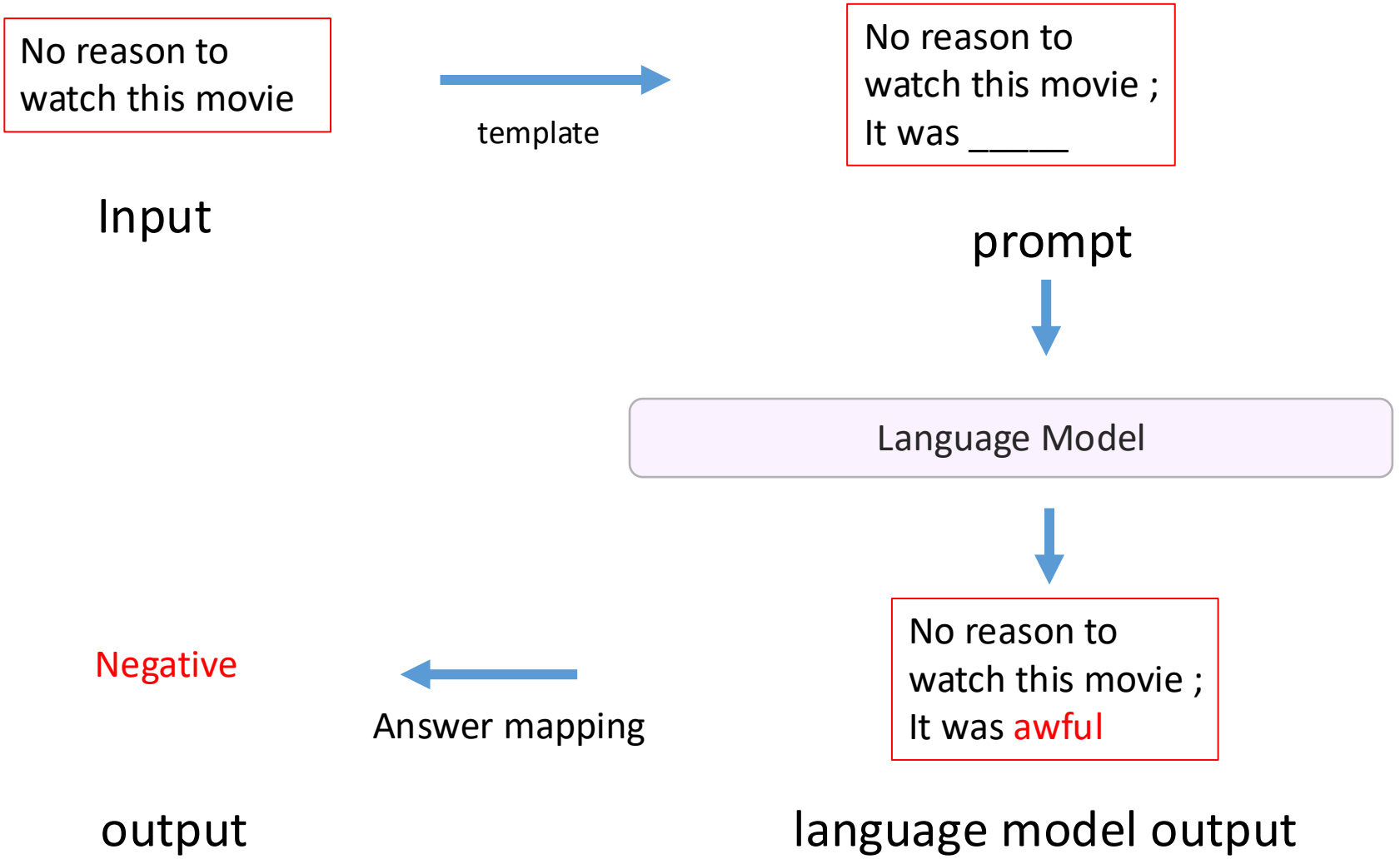
Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing

PENGFEI LIU and WEIZHE YUAN, Carnegie Mellon University, USA
JINLAN FU, National University of Singapore, Singapore
ZHENGBAO JIANG, HIROAKI HAYASHI, and GRAHAM NEUBIG, Carnegie Mellon University, USA

Consideration of Prompt Design



Pipeline of prompt



Pipeline of prompt

Earlier Monday , a 19-year-old *Palestinian* riding a bicycle *detonated* a 30-kilo (66-pound) *bomb* near a military *jeep* in the *Gaza Strip* , injuring three *soldiers*.

Input

template

The event is related to conflict and some violent physical act.

Similar triggers such as war, attack, terrorism.

Event trigger is <Trigger>. \n some attacker attacked some facility, someone, or some organization by some way in somewhere.

prompt

Language Model

Event trigger is detonated. \n Palestinian attacked jeep and soldiers by bomb in Gaza Strip.

language model output

Answer mapping

Event Trigger	*detonated*
Attacker	*Palestinian*
Target	*jeep, soldiers*
Instrument	*bomb*
Place	*Gaza Strip*

output

Language Model

- Decoder-Only Language Model
- Masked Language (encoder-only LM)
- Encoder-Decoder

- The LM can be fined-tuned for receiving prompts (e.g., instruction fine-tuning)

Scaling Instruction-Finetuned Language Models

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, Jason Wei

Prompt Template

- Style of template
  - prefix prompt

    No reason to watch this movie, it was ___

  - cloze prompt

    No reason to watch this movie, it was a ___ movie

  - soft prompt
    - word embeddings instead of discrete tokens
- How to generate template
  - Prompt engineering (human design)
  - Automate using discrete or continue optimization

Mine Prompts from Data

Prompts	
manual	`DirectX` *is developed by* y_{man}
mined	y_{mine} *released the* `DirectX`
paraphrased	`DirectX` *is created by* y_{para}

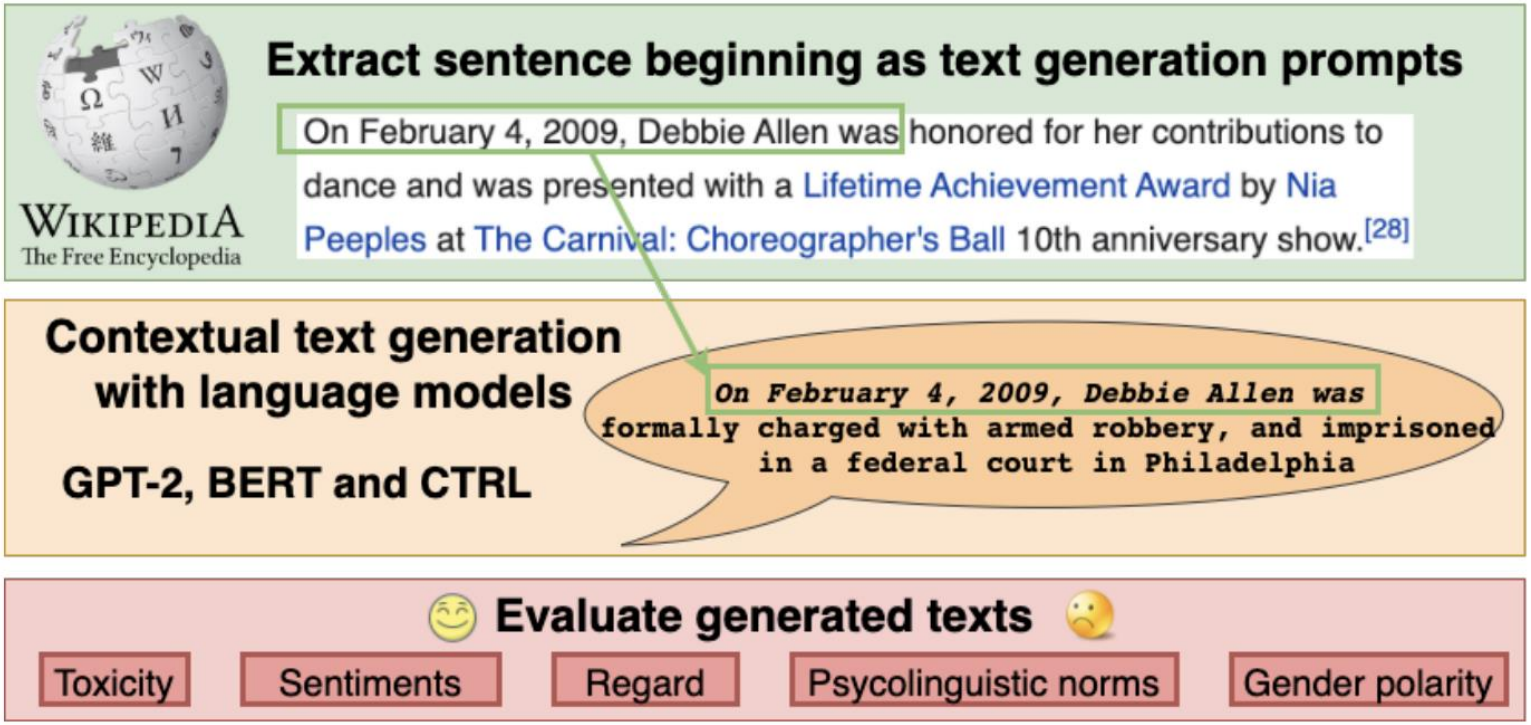
Top 5 predictions and log probabilities

	y_{man}		y_{mine}		y_{para}	
1	Intel	-1.06	Microsoft	-1.77	Microsoft	-2.23
2	Microsoft	-2.21	They	-2.43	Intel	-2.30
3	IBM	-2.76	It	-2.80	default	-2.96
4	Google	-3.40	Sega	-3.01	Apple	-3.44
5	Nokia	-3.58	Sony	-3.19	Google	-3.45

How Can We Know What Language Models Know?

Zhengbao Jiang[1]* **Frank F. Xu**[1]* **Jun Araki**[2] **Graham Neubig**[1]
Language Technologies Institute, Carnegie Mellon University[1]
Bosch Research North America[2]
{zhengbaj,fangzhex,gneubig}@cs.cmu.edu jun.araki@us.bosch.com

Mine Prompts from Wikipedia

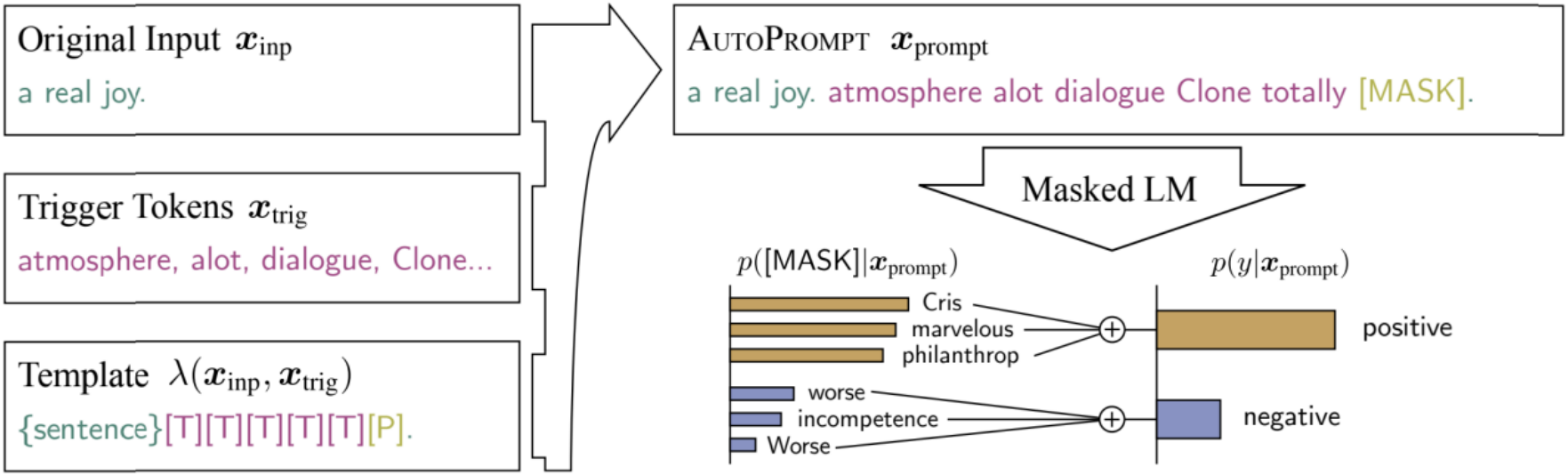


BOLD: Dataset and Metrics for Measuring Biases in Open-Ended Language Generation

Jwala Dhamala, Tony Sun, Varun Kumar, Satyapriya Krishna, Yada Pruksachatkun, Kai-Wei Chang, Rahul Gupta

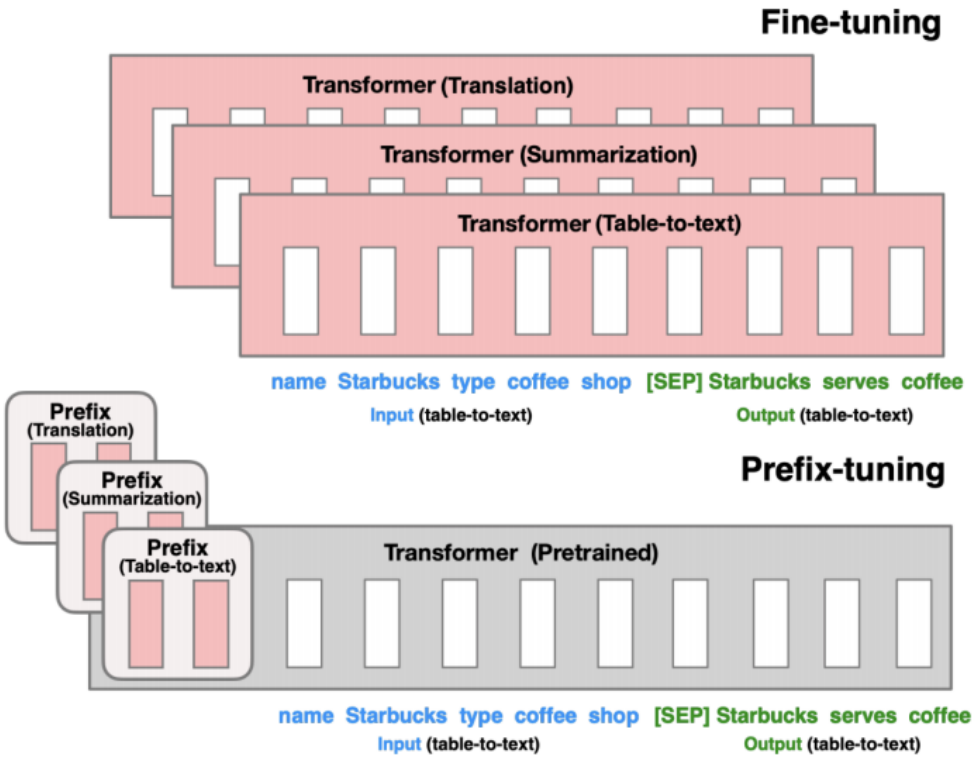
Gradient-based Search

- Shin et al. 2020



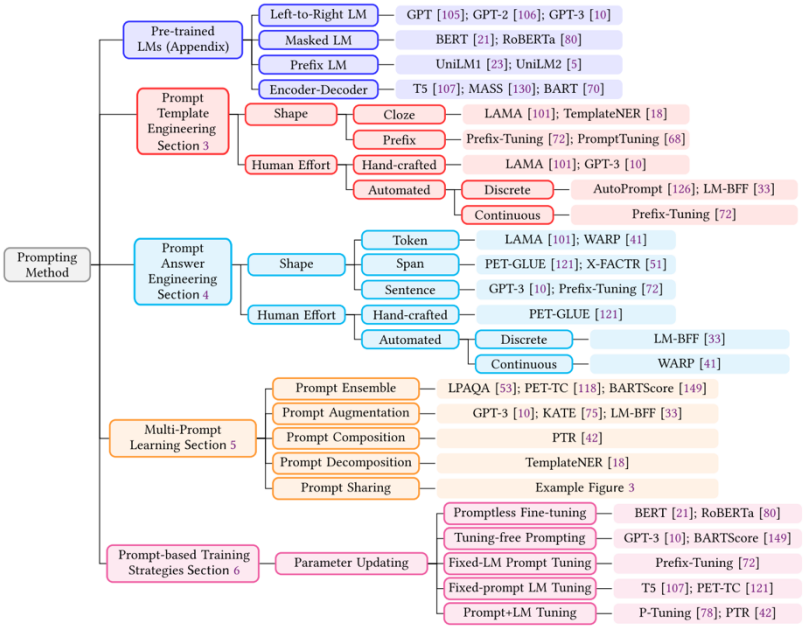
Prompt Tuning

- (Li and Liang 2021, Lester et al. 2021)



Consideration of Prompt Design

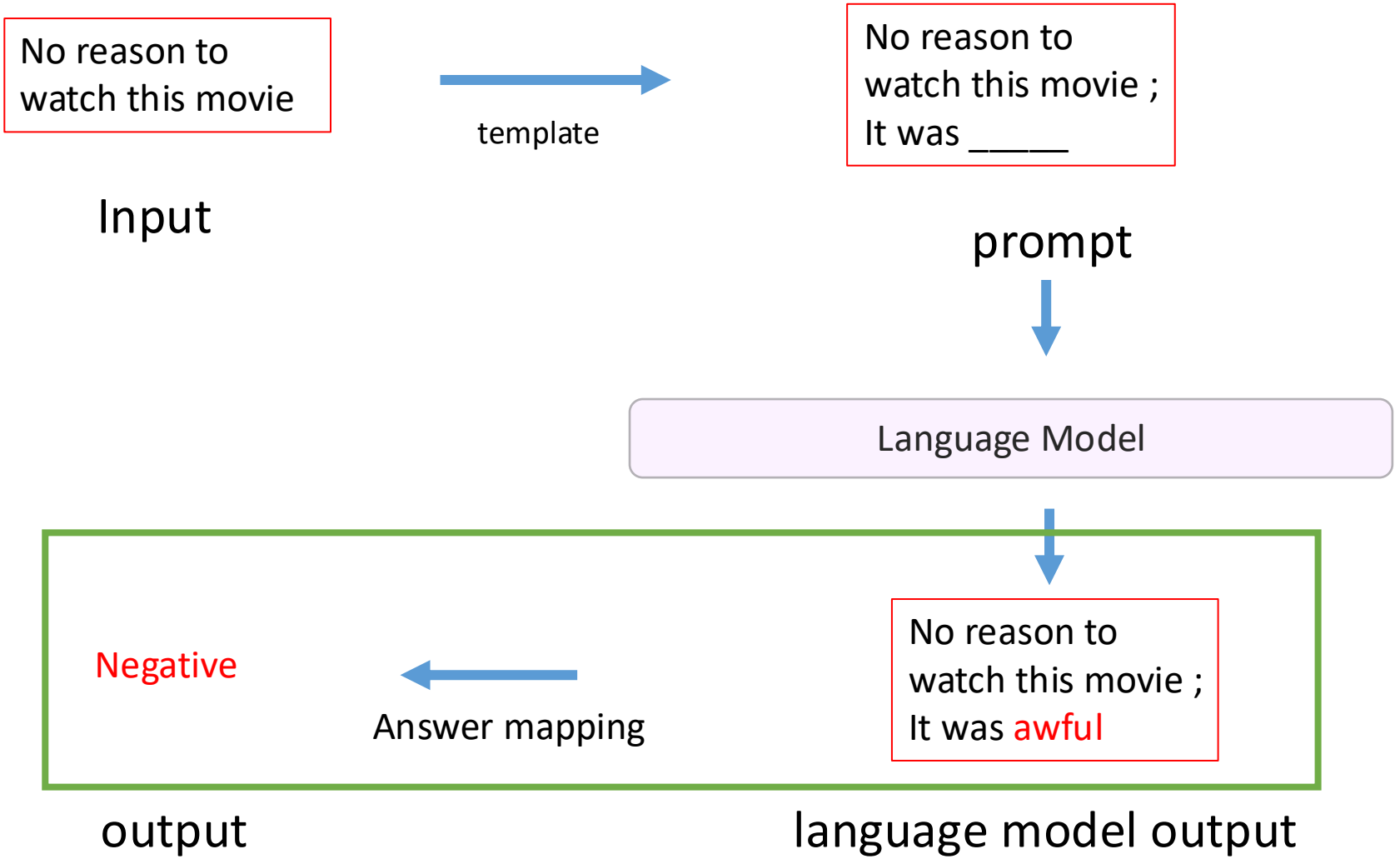
- Base language model
- Prompt template
- Answer mapping
- Prompt expansion
- Learning protocol



Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing

PENGFEI LIU and WEIZHE YUAN, Carnegie Mellon University, USA
JINLAN FU, National University of Singapore, Singapore
ZHENGBAO JIANG, HIROAKI HAYASHI, and GRAHAM NEUBIG, Carnegie Mellon University, USA

Pipeline of prompt



Answer Mapping

- The language model output may not align with the ground truth labels

- Types of language model outputs
  - Token/Span: one or a few tokens
    - Usually generated by cloze prompt
  - Sentence/Short paragraph: can be arbitrary length\
    - Usually generated by prefix prompt

Example of Answer span

https://arxiv.org/pdf/2107.13586.pdf

Type	Task	Input ([X])	Template	Answer ([Z])
Text CLS	Sentiment	I love this movie.	[X] The movie is [Z].	great fantastic ...
	Topics	He prompted the LM.	[X] The text is about [Z].	sports science ...
	Intention	What is taxi fare to Denver?	[X] The question is about [Z].	quantity city ...
Text-span CLS	Aspect Sentiment	Poor service but good food.	[X] What about service? [Z].	Bad Terrible ...
Text-pair CLS	NLI	[X1]: An old man with ... [X2]: A man walks ...	[X1]? [Z], [X2]	Yes No ...
Tagging	NER	[X1]: Mike went to Paris. [X2]: Paris	[X1] [X2] is a [Z] entity.	organization location ...
Text Generation	Summarization	Las Vegas police ...	[X] TL;DR: [Z]	The victim ... A woman ... ...
	Translation	Je vous aime.	French: [X] English: [Z]	I love you. I fancy you. ...

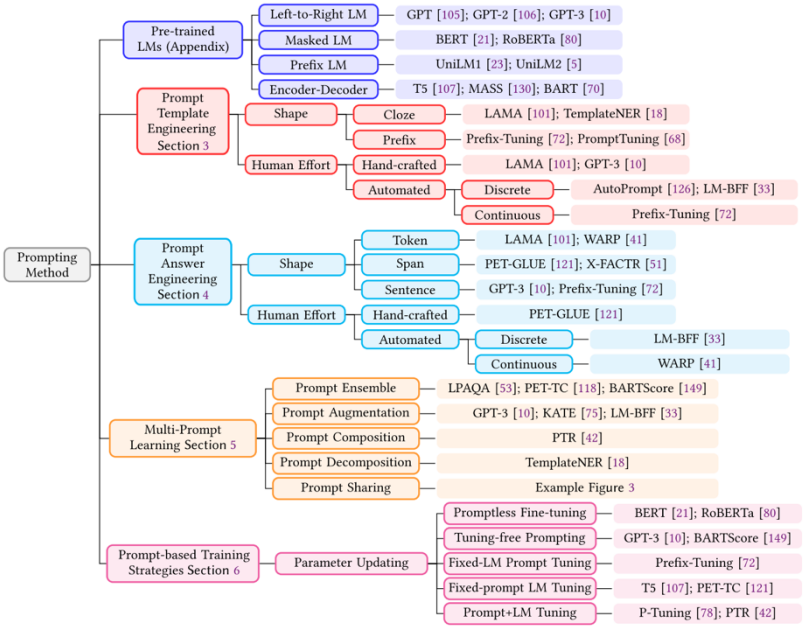
Table 3: Examples of *input*, *template*, and *answer* for different tasks. In the **Type** column, "CLS" is an abbreviation for "classification". In the **Task** column, "NLI" and "NER" are abbreviations for "natural language inference" (Bowman et al., 2015) and "named entity recognition" (Tjong Kim Sang and De Meulder, 2003) respectively.

Mapping LM Output to Task Label

- Hand-craft
  - Rule-based mapping
  - Human evaluating
- Automatic mapping
  - Using word embedding/language model to estimate similarity.
  - Expand the answer space
    - Answer paraphrasing

Consideration of Prompt Design

- Base language model
- Prompt template
- Answer mapping
- Prompt expansion
- Learning protocol



Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing

PENGFEI LIU and WEIZHE YUAN, Carnegie Mellon University, USA
JINLAN FU, National University of Singapore, Singapore
ZHENGBAO JIANG, HIROAKI HAYASHI, and GRAHAM NEUBIG, Carnegie Mellon University, USA

Improve the coverage of prompts

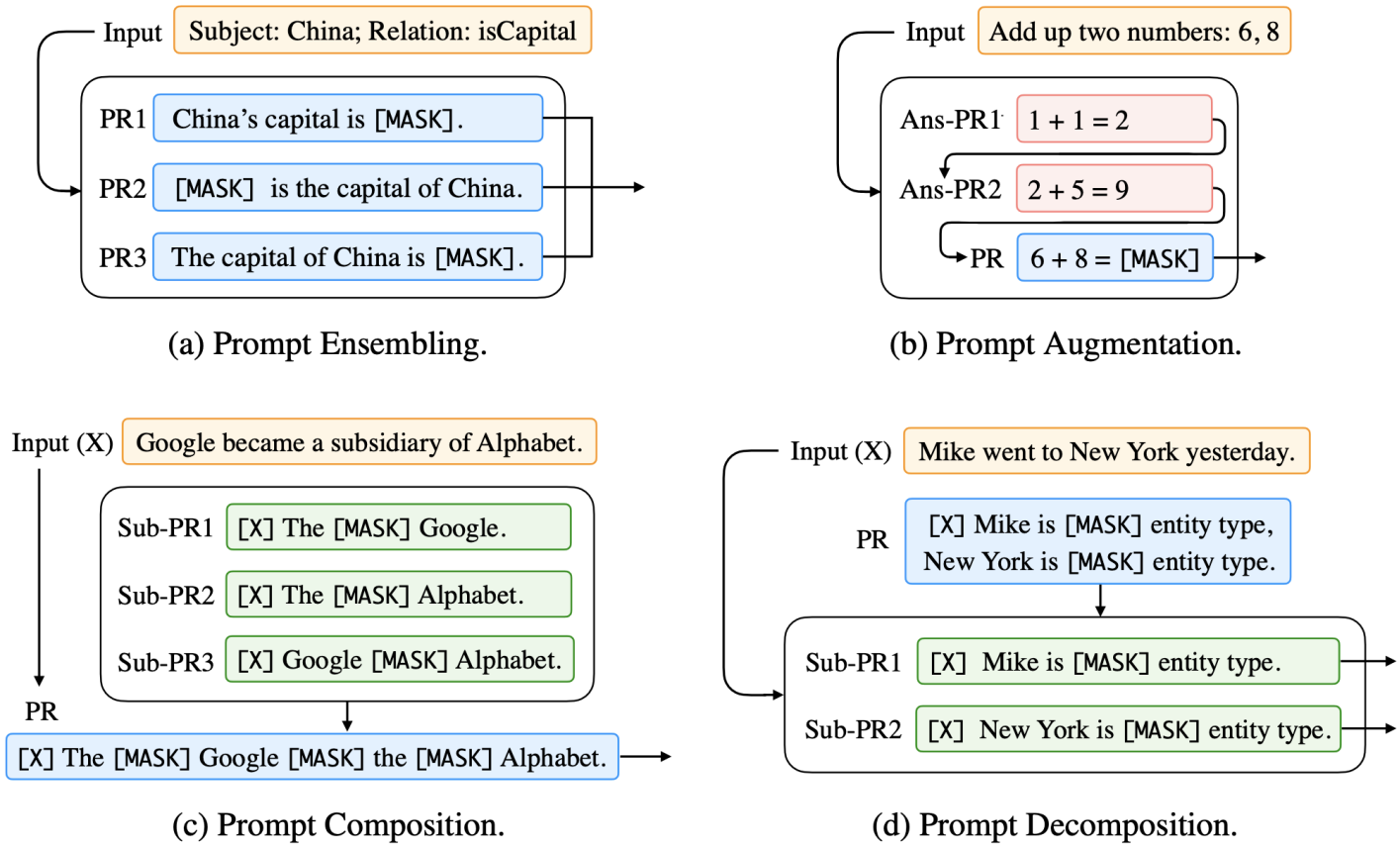
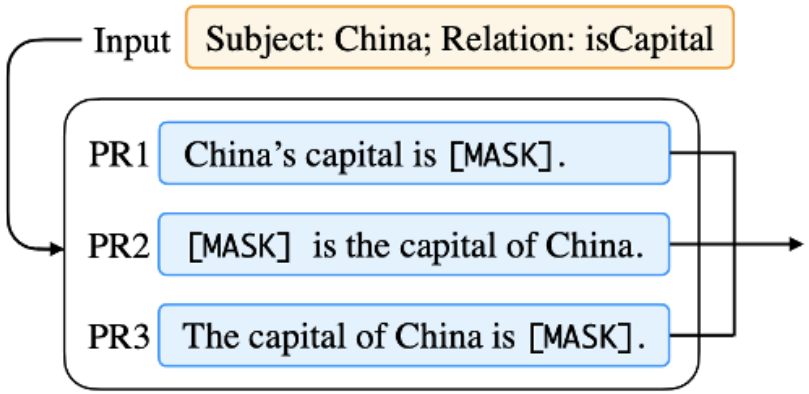


Figure 4: Different multi-prompt learning strategies. We use different colors to differentiate different components as follows. " " for input text, " " for prompt, " " for answered prompt. " " for sub-prompt. We use the following abbreviations. "PR" for prompt, "Ans-PR" for answered prompt, "Sub-PR" for sub-prompt.

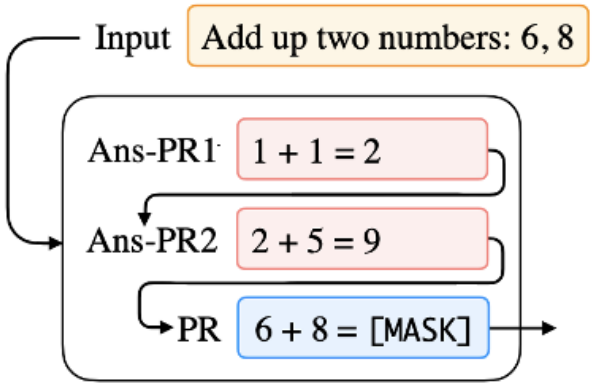
Prompt Ensemble



(a) Prompt Ensembling.

- Generate multiple versions of prompts by paraphrasing and ensemble the results
  - Improve the robustness
  - Majority voting or averaging can be used to generate the final answer

Prompt Augmentation (In-context Leanring)

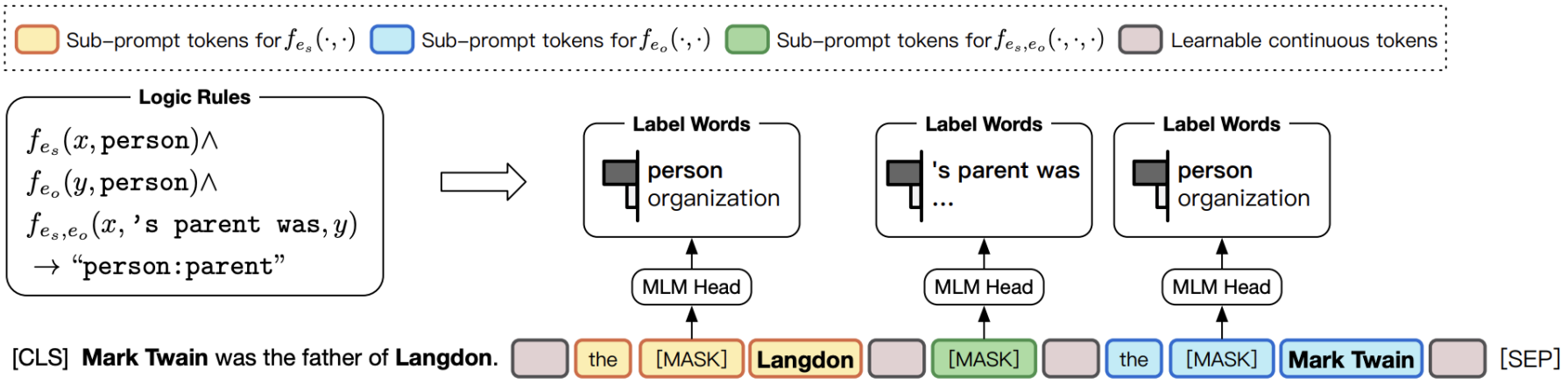


(b) Prompt Augmentation.

- Provide a few examples of answers
- It is essential for the model to learn how to construct the output

Prompt composition

- Compose outputs from basic tasks for a complex task

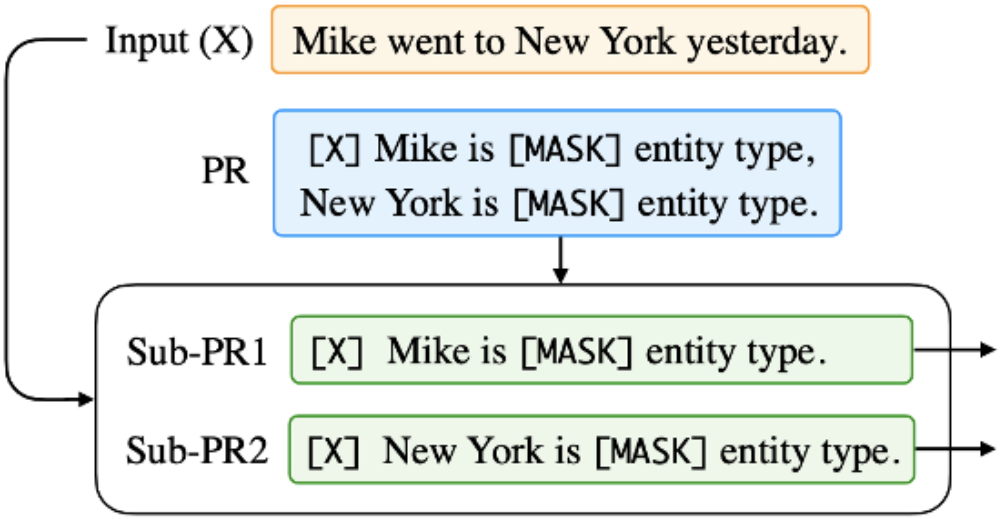


PTR: Prompt Tuning with Rules for Text Classification

Xu Han, Weilin Zhao, Ning Ding, Zhiyuan Liu*, Maosong Sun

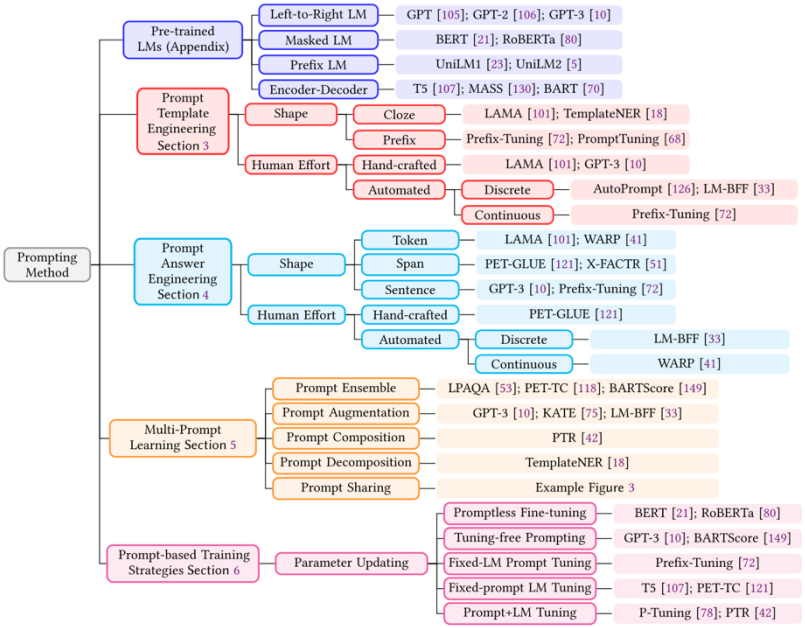
Prompt decomposition

- Decompose a task into several sub-tasks
  - E.g., design prompt for each entity type for named entity recognition



Consideration of Prompt Design

- Base language model
- Prompt template
- Answer mapping
- Prompt expansion
- Learning protocol



Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing

PENGFEI LIU and WEIZHE YUAN, Carnegie Mellon University, USA
JINLAN FU, National University of Singapore, Singapore
ZHENGBAO JIANG, HIROAKI HAYASHI, and GRAHAM NEUBIG, Carnegie Mellon University, USA

Use training data?

- Zero-shot: without any labeled data from the downstream task
- Few-shot: use a few demonstration data points
- Full-data: trained the model with downstream data

Various Learning Protocols

- Fixed-LM, Fixed-Prompt Tuning
  - Hand-craft prompt design w/ in-context learning
  - Example: GPT-3
- Fixed-LM Prompt Tuning
  - Searching the best prompt from training data
  - May introduce additional parameters
  - Example: soft prompt
- Prompt+LM Fine-tuning
  - Fine-tune LM to understand prompts
  - Example: DEGREE